# Project - Low Level Design

# On

# Multi-Agent Education System

Course Name: DevOps

***Institution Name:*** Medicaps University – Datagami Skill Based Course

*Student Name(s) & Enrolment Number(s):*

| Sr no | Student Name | Enrolment Number |
|---|---|---|
| 1. | SHREYA MAHODAY | EN22CS301930 |
| 2. | SARTHAK PATEL | EN22CS301877 |
| 3. | SAKSHI JAIN | EN22CS301845 |
| 4. | SIYA KUSUMAKAR | EN22CS301960 |
| 5. | SHREYA THAKKAR | EN22CS301933 |

*Group Name: 09D8*

*Project Number: DO-44*

*Industry Mentor Name:*

*University Mentor Name: Prof Avnesh Kumar Joshi*

*Academic Year: 2025 - 2026*

# Table of Contents

1. Introduction.
   1.1. Scope of the document.
   1.2. Intended Audience
   1.3. System overview.

2. System Design.
   2.1. Application Design
   2.2. Process Flow.
   2.3. Information Flow.
   2.4. Components Design
   2.5. Key Design Considerations
   2.6. API Catalogue.

3. References

# 1. Introduction

## 1.1. Scope of the Document

This document provides the low-level design of the EduAI web application deployment. It focuses on the DevOps implementation used to deploy the application using Docker and Kubernetes.

The document explains how the application is containerized, stored in a container registry, and deployed in a Kubernetes cluster. It also covers system components, process flow, and design considerations for scalability, availability, and reliability.

## 1.2. Intended Audience

This document is intended for:

- Developers
- DevOps Engineers
- Project Evaluators
- Students and Learners

## 1.3. System Overview

The EduAI application is a Streamlit-based web application that is deployed using modern DevOps practices instead of running locally.

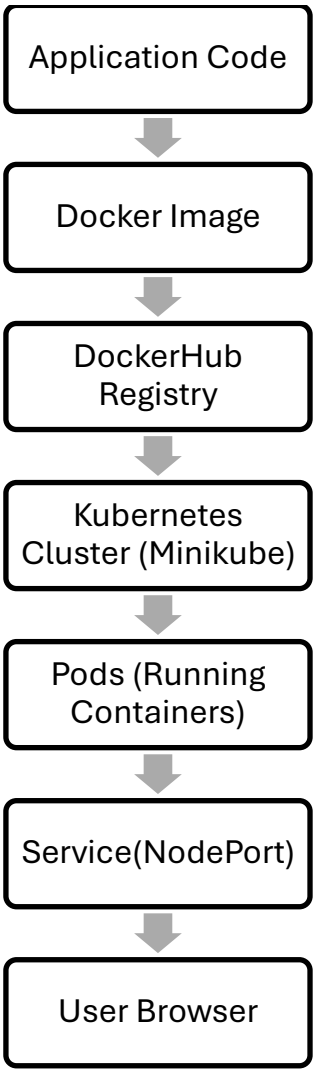
Traditionally, applications are run using commands like:

python app.py

However, this approach has limitations such as dependency issues, lack of scalability, and no fault tolerance.

To overcome these issues, the system uses containerization and orchestration.

**System Workflow**




# 2. System Design

## 2.1. Application Design

The application is built using Streamlit and is containerized using Docker.

The Docker container includes:

- Python 3.10 runtime
- Required dependencies (libraries)
- Application source code

A Dockerfile is used to define how the container image is built.

**Dockerfile Overview**

```
FROM python:3.10
WORKDIR /app
COPY . .
RUN pip install -r requirements.txt
CMD streamlit run app.py
```

This ensures that the application runs consistently across all environments.

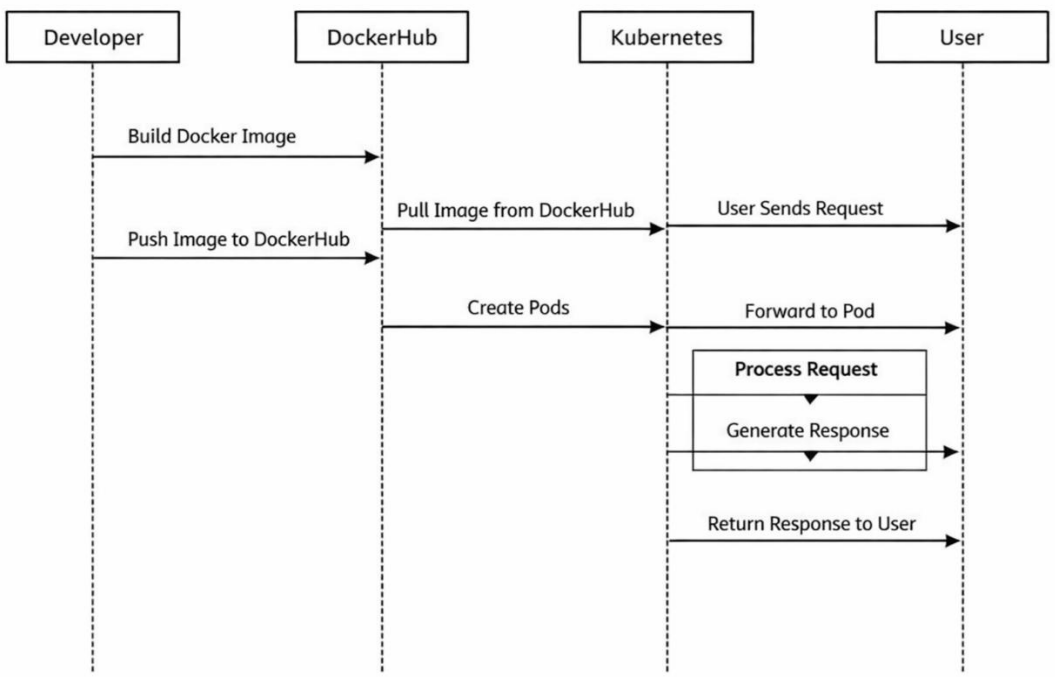
**Container Structure**

```
| Docker Container |
| Python 3.10      |
| Libraries        |
| Streamlit App    |
```

## 2.2. Process Flow

The deployment process follows a structured pipeline:

1. Developer writes application code
2. Docker image is built using Dockerfile
3. Image is pushed to DockerHub
4. Kubernetes pulls the image
5. Pods are created
6. Service exposes the application
7. User accesses the application via browser




## 2.3. Information Flow

- Developer builds the Docker image from the application code.
- The Docker image is pushed to DockerHub.
- Kubernetes pulls the image and creates pods using Deployment.
- A Service is used to expose the application to users.
- User sends a request through the browser.
- The Service forwards the request to one of the pods.
- The container processes the request and generates a response.
- The response is sent back to the user through the Service.

## 2.4. Components Design

The system consists of the following components:

**1. Docker**

- Used for containerizing the application
- Packages code, dependencies, and runtime environment

**2. DockerHub**

- Acts as a container registry
- Stores and distributes Docker images

**3. Kubernetes**

- Manages container deployment
- Handles scaling and fault tolerance

**4. Deployment**

- Ensures the desired number of pods are running
- Manages updates and rollbacks

**5. Pods**

- Smallest deployable units in Kubernetes
- Run containerized applications

**6. Service (NodePort)**

- Exposes the application externally
- Routes traffic to pods

## 2.5. Key Design Considerations

The system is designed based on the following principles:

- **Scalability**: The number of pods can be increased or decreased based on demand.
- **High Availability**: Multiple replicas ensure that the application remains available even if one pod fails.

- **Portability**: Docker ensures that the application runs consistently across different environments.
- **Fault Tolerance**: Kubernetes automatically restarts failed containers.
- **Zero Downtime Deployment**: Rolling updates allow new versions to be deployed without stopping the application.

### 2.6 API Catalogue

- The application does not use external APIs.
- It runs as a standalone Streamlit application.
- The application is exposed on port 8501 inside the container.

## 3. References

- Docker Official Documentation
- Kubernetes Official Documentation
- Minikube Documentation